

23. live 風補正ノート

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	7
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 list、dict、deque、NumPy 配列、pandas DataFrame	8
6.9 rclpy、rcl、rmw、DDS/RTSPS の層	8
6.10 ROS graph、Node、topic、service、action、parameter	8
6.11 callback、timer、Executor、spin、Callback Group	8
6.12 rqt_graph、ros2 CLI、rosviz をいつ使うか	9
6.13 天候、route、補間、時刻、単位	9
6.14 freshness、filter、guard、fallback、fail-safe	10
6.15 制御、状態、入力、モデル予測制御 MPC	10

7 関数・クラスを上から順に解説	10
7.1 L16 関数 <code>_timestamp_ns</code>	10
7.2 L20 クラス <code>WindCorrectionNode</code>	11
7.3 L21 関数 <code>WindCorrectionNode.__init__</code>	12
7.4 L64 関数 <code>WindCorrectionNode._on_headwind</code>	15
7.5 L67 関数 <code>WindCorrectionNode._on_s</code>	16
7.6 L70 関数 <code>WindCorrectionNode._load_base</code>	17
7.7 L91 関数 <code>WindCorrectionNode._interp_headwind_now</code>	18
7.8 L105 関数 <code>WindCorrectionNode._step</code>	19
7.9 L154 関数 <code>main</code>	22
7.10 設定値と入力パラメータ	23
7.11 ROS topic I/O	23
8 処理の流れ	24
9 このファイルの位置づけ	24

1 対象

項目	内容
ファイル	mpc_solarcar/wind_correction_node.py
ソース SHA-256	065128cbb44604d15e3d04030f00c23607abfa9faf13ebb66e08835897217593
種別	Python
区分	runtime node
役割	観測 headwind と現在距離から raw forecast CSV を補正し、corrected forecast CSV を書き出す。
起動文脈	live_wifi で planner 入力の風を上書きする前処理。

2 このファイルを読む理由

観測 headwind と現在距離から raw forecast CSV を補正し、corrected forecast CSV を書き出す。

- planner は /weather/headwind_corrected_ms を直接読むのではない。
- corrected CSV を mpc_node が再読込して効かせる。

3 呼び出し関係

3.1 どこから呼ばれるか

- launch/solar_race_live_wifi.launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/mpc_node.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
return int(pd.Timestamp(value).value)

class WindCorrectionNode(Node):
    def __init__(self) -> None:
        super().__init__("wind_correction_node")
        self.declare_parameter("forecast_csv", "")
        self.declare_parameter("corrected_forecast_csv", "outputs/runtime/
live_forecast_corrected.csv")
        self.declare_parameter("forecast_time_tz", "Australia/Darwin")
```

```

self.declare_parameter("publish_period_sec", 30.0)
self.declare_parameter("measurement_sigma_ms", 1.0)
self.declare_parameter("correlation_distance_km", 300.0)
self.declare_parameter("fallback_correlation_time_h", 3.0)
self.declare_parameter("forecast_sigma0_ms", 1.5)
self.declare_parameter("forecast_variance_growth_per_hour", 0.05)
self.declare_parameter("planning_quantile", 0.5)
self.declare_parameter("confidence_z", 1.96)
self.declare_parameter("min_sigma_ms", 0.2)
self.declare_parameter("preferred_source", "auto")
self.declare_parameter("use_exp_distance_decay", True)

self.forecast_csv = Path(str(self.get_parameter("forecast_csv").value or "")).
expanduser()

```

5 主要構造の読み取り

主要クラスは WindCorrectionNode。主要関数は main。ROS パラメータ宣言は 14 件。ROS I/O は publisher 2 件、subscription 2 件。

5.1 主要クラス・関数

行	種別	名前	補足
20	class	WindCorrectionNode	bases: Node
16	function	_timestamp_ns	args: value
21	function	__init__	args: self
64	function	_on_headwind	args: self, msg
67	function	_on_s	args: self, msg
70	function	_load_base	args: self
91	function	_interp_headwind_now	args: self, now_utc
105	function	_step	args: self
154	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
16	関数 _timestamp_ns を定義する。
20	クラス WindCorrectionNode を定義する。
154	関数 main を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	from __future_ _import annotations	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。

3	<code>importmath</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L52, L53, L111, L121, L122, L130, L131, L135。
4	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L72。
5	<code>fromdatetetimeimportdatetetime, timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L91, L109, L128。
6	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L38, L39。
7	<code>fromstatisticsimportNormalDist</code>	statistics から NormalDist を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L50。
9	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L17, L77, L79, L96, L119, L128, L140, L147。
10	<code>importrcclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L155, L157, L159。
11	<code>fromrcclpy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L20。
13	<code>fromstd_msgs. msgimportFloat32,String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L57, L58, L59, L60, L64, L67, L115, L151。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体を作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self._step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 '(N)」、候補集団なら '(population, N)となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.9 rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC 内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executor の実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.10 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.11 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。


```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.12 rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.13 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.14 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.15 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](https://docs.scipy.org/doc/scipy/reference/optimize.minimize.html)

7 関数・クラスを上から順に解説

7.1 L16 関数 _timestamp_ns

- 行範囲: L16–L17
- 所属: module 直下
- 定義: `_timestamp_ns(value) -> int`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `int`
- 戻り値の要点: `int(pd.Timestamp(value).value)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `int(pd.Timestamp(value).value)` を返す。

代表コード断片

```
def _timestamp_ns(value) -> int:
    return int(pd.Timestamp(value).value)
```

7.2 L20 クラス WindCorrectionNode

- 行範囲: L20–L151
- 所属: module 直下
- 定義: `WindCorrectionNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_on_headwind` を定義する。
3. 関数 `_on_s` を定義する。
4. 関数 `_load_base` を定義する。
5. 関数 `_interp_headwind_now` を定義する。
6. 関数 `_step` を定義する。

代表コード断片

```
class WindCorrectionNode(Node):
    def __init__(self) -> None:
        super().__init__("wind_correction_node")
        self.declare_parameter("forecast_csv", "")
        self.declare_parameter("corrected_forecast_csv", "outputs/runtime/
live_forecast_corrected.csv")
        self.declare_parameter("forecast_time_tz", "Australia/Darwin")
        self.declare_parameter("publish_period_sec", 30.0)
        self.declare_parameter("measurement_sigma_ms", 1.0)
        self.declare_parameter("correlation_distance_km", 300.0)
        self.declare_parameter("fallback_correlation_time_h", 3.0)
        self.declare_parameter("forecast_sigma0_ms", 1.5)
        self.declare_parameter("forecast_variance_growth_per_hour", 0.05)
        self.declare_parameter("planning_quantile", 0.5)
        self.declare_parameter("confidence_z", 1.96)
        self.declare_parameter("min_sigma_ms", 0.2)
        self.declare_parameter("preferred_source", "auto")
        self.declare_parameter("use_exp_distance_decay", True)

        self.forecast_csv = Path(str(self.get_parameter("forecast_csv").value or "")).
expanduser()
        self.corrected_csv = Path(str(self.get_parameter("corrected_forecast_csv").value or
"")).expanduser()
        self.forecast_time_tz = str(self.get_parameter("forecast_time_tz").value or "
Australia/Darwin")
        self.measurement_sigma = max(0.0, float(self.get_parameter("measurement_sigma_ms").
value))
        self.correlation_distance_km = max(1.0, float(self.get_parameter("
correlation_distance_km").value))
        self.correlation_time_h = max(0.25, float(self.get_parameter("
fallback_correlation_time_h").value))
        self.forecast_sigma0 = max(0.0, float(self.get_parameter("forecast_sigma0_ms").value
))
        self.variance_growth = max(0.0, float(self.get_parameter("
forecast_variance_growth_per_hour").value))
        self.planning_quantile = min(0.999, max(0.001, float(self.get_parameter("
planning_quantile").value)))
        self.confidence_z = max(0.1, float(self.get_parameter("confidence_z").value))
        self.min_sigma = max(0.0, float(self.get_parameter("min_sigma_ms").value))
        self.use_exp_distance_decay = bool(self.get_parameter("use_exp_distance_decay").
value)
        self.quantile_z = float(NormalDist().inv_cdf(self.planning_quantile))

        self.latest_headwind = math.nan
        self.latest_s_km = math.nan
        self.latest_bias = 0.0
    ...
```

7.3 L21 関数 WindCorrectionNode.__init__

- 行範囲: L21-L62
- 所属: WindCorrectionNode
- 定義: __init__(self)->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: NormalDist, Path, __init__, bool, create_publisher, create_subscription, create_timer, declare_parameter, expanduser, float, get_logger, get_parameter

- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.on_headwind`, `self.on_s`, `self.step`, `self.corrected_csv`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.forecast_csv`, `self.get_logger`, `self.get_parameter`, `self.planning_quantile`
- 更新する主なインスタンス属性: `self.base_df`, `self.base_mtime`, `self.confidence_z`, `self.corrected_csv`, `self.correlation_distance_km`, `self.correlation_time_h`, `self.forecast_csv`, `self.forecast_sigma0`, `self.forecast_time_tz`, `self.latest_bias`, `self.latest_headwind`, `self.latest_s_km`, `self.measurement_sigma`, `self.min_sigma`, `self.planning_quantile`, `self.pub_headwind`, `self.pub_status`, `self.quantile_z`, `self.timer`, `self.use_exp_distance_decay`, `self.variance_growth`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。
13. `self.declare_parameter(...)` を実行する。
14. `self.declare_parameter(...)` を実行する。
15. `self.declare_parameter(...)` を実行する。
16. `self.forecast_csv` に `Path(str(self.get_parameter('forecast_csv').value or '').expanduser())` の結果を代入する。
17. `self.corrected_csv` に `Path(str(self.get_parameter('corrected_forecast_csv').value or '').expanduser())` の結果を代入する。
18. `self.forecast_time_tz` に `str(self.get_parameter('forecast_time_tz').value or 'Australia/Darwin')` の結果を代入する。

19. self.measurement_sigma に max(0.0, float(self.get_parameter('measurement_sigma_ms').value)) の結果を代入する。
20. self.correlation_distance_km に max(1.0, float(self.get_parameter('correlation_distance_km').value)) の結果を代入する。
21. self.correlation_time_h に max(0.25, float(self.get_parameter('fallback_correlation_time_h').value)) の結果を代入する。
22. self.forecast_sigma0 に max(0.0, float(self.get_parameter('forecast_sigma0_ms').value)) の結果を代入する。
23. self.variance_growth に max(0.0, float(self.get_parameter('forecast_variance_growth_per_hour').value)) の結果を代入する。
24. self.planning_quantile に min(0.999, max(0.001, float(self.get_parameter('planning_quantile').value))) の結果を代入する。
25. self.confidence_z に max(0.1, float(self.get_parameter('confidence_z').value)) の結果を代入する。
26. self.min_sigma に max(0.0, float(self.get_parameter('min_sigma_ms').value)) の結果を代入する。
27. self.use_exp_distance_decay に bool(self.get_parameter('use_exp_distance_decay').value) の結果を代入する。
28. self.quantile_z に float(NormalDist().inv_cdf(self.planning_quantile)) の結果を代入する。
29. self.latest_headwind に math.nan の結果を代入する。
30. self.latest_s_km に math.nan の結果を代入する。
31. self.latest_bias に 0.0 の結果を代入する。
32. self.base_df に None の結果を代入する。
33. self.base_mtime に None の結果を代入する。
34. self.pub_headwind に self.create_publisher(Float32, '/weather/headwind_corrected_ms', 10) の結果を代入する。
35. self.pub_status に self.create_publisher(String, '/weather/wind_correction_status', 10) の結果を代入する。
36. self.create_subscription(...) を実行する。
37. self.create_subscription(...) を実行する。
38. self.timer に self.create_timer(max(1.0, float(self.get_parameter('publish_period_sec').value)), self_step) の結果を代入する。
39. self.get_logger().info(...) を実行する。

代表コード断片

```
def __init__(self) -> None:
    super().__init__("wind_correction_node")
    self.declare_parameter("forecast_csv", "")
    self.declare_parameter("corrected_forecast_csv", "outputs/runtime/
live_forecast_corrected.csv")
    self.declare_parameter("forecast_time_tz", "Australia/Darwin")
    self.declare_parameter("publish_period_sec", 30.0)
    self.declare_parameter("measurement_sigma_ms", 1.0)
    self.declare_parameter("correlation_distance_km", 300.0)
    self.declare_parameter("fallback_correlation_time_h", 3.0)
    self.declare_parameter("forecast_sigma0_ms", 1.5)
    self.declare_parameter("forecast_variance_growth_per_hour", 0.05)
    self.declare_parameter("planning_quantile", 0.5)
```

```

self.declare_parameter("confidence_z", 1.96)
self.declare_parameter("min_sigma_ms", 0.2)
self.declare_parameter("preferred_source", "auto")
self.declare_parameter("use_exp_distance_decay", True)

self.forecast_csv = Path(str(self.get_parameter("forecast_csv").value or "")).
expanduser()
self.corrected_csv = Path(str(self.get_parameter("corrected_forecast_csv").value or
"")).expanduser()
self.forecast_time_tz = str(self.get_parameter("forecast_time_tz").value or "
Australia/Darwin")
self.measurement_sigma = max(0.0, float(self.get_parameter("measurement_sigma_ms").
value))
self.correlation_distance_km = max(1.0, float(self.get_parameter("
correlation_distance_km").value))
self.correlation_time_h = max(0.25, float(self.get_parameter("
fallback_correlation_time_h").value))
self.forecast_sigma0 = max(0.0, float(self.get_parameter("forecast_sigma0_ms").value
))
self.variance_growth = max(0.0, float(self.get_parameter("
forecast_variance_growth_per_hour").value))
self.planning_quantile = min(0.999, max(0.001, float(self.get_parameter("
planning_quantile").value)))
self.confidence_z = max(0.1, float(self.get_parameter("confidence_z").value))
self.min_sigma = max(0.0, float(self.get_parameter("min_sigma_ms").value))
self.use_exp_distance_decay = bool(self.get_parameter("use_exp_distance_decay").
value)
self.quantile_z = float(NormalDist().inv_cdf(self.planning_quantile))

self.latest_headwind = math.nan
self.latest_s_km = math.nan
self.latest_bias = 0.0
self.base_df = None

...

```

7.4 L64 関数 WindCorrectionNode._on_headwind

- 行範囲: L64–L65
- 所属: WindCorrectionNode
- 定義: `_on_headwind(self, msg: Float32) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.latest_headwind`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.latest_headwind` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_headwind(self, msg: Float32) -> None:
    self.latest_headwind = float(msg.data)
```

7.5 L67 関数 `WindCorrectionNode._on_s`

- 行範囲: L67-L68
- 所属: `WindCorrectionNode`
- 定義: `_on_s(self, msg: Float32) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.latest_s_km`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.latest_s_km` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_s(self, msg: Float32) -> None:
    self.latest_s_km = float(msg.data)
```


7.6 L70 関数 `WindCorrectionNode._load_base`

- 行範囲: L70–L89
- 所属: `WindCorrectionNode`
- 定義: `_load_base(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `getattr`, `getmtime`, `read_csv`, `to_datetime`, `tz_convert`, `tz_localize`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `df`, `mtime`, `t`
- 読み取る主なインスタンス属性: `self.base_df`, `self.base_mtime`, `self.forecast_csv`, `self.forecast_time_tz`
- 更新する主なインスタンス属性: `self.base_df`, `self.base_mtime`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 2

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `mtime` に `os.path.getmtime(self.forecast_csv)` の結果を代入する。
3. `Exception` を捕捉した場合:
4. を返す。
5. 条件 `self.base_mtime is not None and mtime <= self.base_mtime and (self.base_df is not None)` を判定し、真なら内部処理を行う。
6. を返す。
7. `df` に `pd.read_csv(self.forecast_csv)` の結果を代入する。
8. 条件 `'time' in df.columns` を判定し、真なら内部処理を行う。
9. `t` に `pd.to_datetime(df['time'], format='mixed', errors='coerce')` の結果を代入する。
10. 条件 `getattr(t.dt, 'tz', None) is None` を判定し、真なら内部処理を行う。
11. 例外処理を伴う `try` ブロックを実行する。
12. `t` に `t.dt.tz_localize(self.forecast_time_tz, ambiguous='NaT', nonexistent='NaT').dt.tz_convert('UTC')` の結果を代入する。
13. `Exception` を捕捉した場合:

14. `t` に `t.dt.tz_localize('UTC')` の結果を代入する。

15. 上の条件が偽の場合:

16. `t` に `t.dt.tz_convert('UTC')` の結果を代入する。

17. `df['time']` に `t` の結果を代入する。

18. `self.base_df` に `df` の結果を代入する。

19. `self.base_mtime` に `mtime` の結果を代入する。

代表コード断片

```
def _load_base(self) -> None:
    try:
        mtime = os.path.getmtime(self.forecast_csv)
    except Exception:
        return
    if self.base_mtime is not None and mtime <= self.base_mtime and self.base_df is not None:
        return
    df = pd.read_csv(self.forecast_csv)
    if "time" in df.columns:
        t = pd.to_datetime(df["time"], format="mixed", errors="coerce")
        if getattr(t.dt, "tz", None) is None:
            try:
                t = t.dt.tz_localize(self.forecast_time_tz, ambiguous="NaT", nonexistent="NaT").dt.tz_convert("UTC")
            except Exception:
                t = t.dt.tz_localize("UTC")
        else:
            t = t.dt.tz_convert("UTC")
        df["time"] = t
    self.base_df = df
    self.base_mtime = mtime
```

7.7 L91 関数 `WindCorrectionNode._interp_headwind_now`

- 行範囲: L91–L103
- 所属: `WindCorrectionNode`
- 定義: `_interp_headwind_now(self, now_utc:datetime)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `any`, `float`, `get`, `int`, `len`, `max`, `min`, `notna`, `searchsorted`
- 戻り値の要点: `0.0 / float(row.get('headwind_ms', 0.0)) / 0.0`
- 主なローカル代入名: `idx`, `row`, `work`
- 読み取る主なインスタンス属性: `self.base_df`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.base_df is None or self.base_df.empty` を判定し、真なら内部処理を行う。
2. `0.0` を返す。
3. `work` に `self.base_df` の結果を代入する。
4. 条件 `'time' in work.columns and work['time'].notna().any()` を判定し、真なら内部処理を行う。
5. `idx` に `int(max(0, min(len(work) - 1, work['time'].searchsorted(pd.Timestamp(now_utc), side='right') - 1)))` の結果を代入する。
6. `row` に `work.iloc[idx]` の結果を代入する。
7. 上の条件が偽の場合:
8. `row` に `work.iloc[0]` の結果を代入する。
9. 例外処理を伴う `try` ブロックを実行する。
10. `float(row.get('headwind_ms', 0.0))` を返す。
11. `Exception` を捕捉した場合:
12. `0.0` を返す。

代表コード断片

```
def _interp_headwind_now(self, now_utc: datetime) -> float:
    if self.base_df is None or self.base_df.empty:
        return 0.0
    work = self.base_df
    if "time" in work.columns and work["time"].notna().any():
        idx = int(max(0, min(len(work) - 1, work["time"].searchsorted(pd.Timestamp(
now_utc), side="right") - 1)))
        row = work.iloc[idx]
    else:
        row = work.iloc[0]
    try:
        return float(row.get("headwind_ms", 0.0))
    except Exception:
        return 0.0
```

7.8 L105 関数 `WindCorrectionNode._step`

- 行範囲: L105–L151
- 所属: `WindCorrectionNode`
- 定義: `_step(self) -> None`
- `docstring`: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: Float32, String, Timedelta, Timestamp, `_interp_headwind_now`, `_load_base`, `abs`, `append`, `astimezone`, `copy`, `exp`, `fillna`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `base`, `base_now`, `corrected`, `corrected_now`, `corrected_values`, `current_s`, `decay`, `df`, `dist_decay`, `ds`, `dt_h`, `i`, `innovation`, `mean`, `now_utc`, `out`, `row`, `sigma`, `sigma_values`, `t_utc`
- 読み取る主なインスタンス属性: `self._interp_headwind_now`, `self._load_base`, `self.base_df`, `self.confidence_z`, `self.corrected_csv`, `self.correlation_distance_km`, `self.correlation_time_h`, `self.forecast_sigma0`, `self.forecast_time_tz`, `self.latest_bias`, `self.latest_headwind`, `self.latest_s_km`, `self.measurement_sigma`, `self.min_sigma`, `self.pub_headwind`, `self.pub_status`, `self.quantile_z`, `self.use_exp_distance_decay`, `self.variance_growth`
- 更新する主なインスタンス属性: `self.latest_bias`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `self._load_base(...)` を実行する。
2. 条件 `self.base_df is None or self.base_df.empty` を判定し、真なら内部処理を行う。
3. を返す。
4. `now_utc` に `datetime.now(timezone.utc)` の結果を代入する。
5. `base_now` に `self._interp_headwind_now(now_utc)` の結果を代入する。
6. 条件 `math.isfinite(self.latest_headwind)` を判定し、真なら内部処理を行う。
7. `innovation` に `float(self.latest_headwind - base_now)` の結果を代入する。
8. `self.latest_bias` に `0.7 * float(self.latest_bias) + 0.3 * innovation` の結果を代入する。
9. `corrected_now` に `base_now + self.latest_bias` の結果を代入する。
10. `self.pub_headwind.publish(...)` を実行する。
11. `df` に `self.base_df.copy()` の結果を代入する。
12. 条件 `'time' not in df.columns` を判定し、真なら内部処理を行う。
13. `df['time']` に `[now_utc + pd.Timedelta(minutes=10 * i) for i in range(len(df))]` の結果を代入する。
14. 条件 `'s_km' not in df.columns` を判定し、真なら内部処理を行う。
15. `df['s_km']` に `float(self.latest_s_km) if math.isfinite(self.latest_s_km) else 0.0` の結果を代入する。
16. `current_s` に `float(self.latest_s_km) if math.isfinite(self.latest_s_km) else float(df['s_km'].iloc[0])` の結果を代入する。
17. `corrected_values` に `[]` の結果を代入する。

18. `sigma_values` に [] の結果を代入する。
19. `df.itertuples(index=False)` を順に走査し、各要素を `row` に入れて処理する。
20. `t_utc` に `getattr(row, 'time')` の結果を代入する。
21. `dt_h` に `max(0.0, (pd.Timestamp(t_utc).to_pydatetime().astimezone(timezone.utc) - now_utc).total_seconds() / 3600.0)` の結果を代入する。
22. `ds` に `abs(float(getattr(row, 's_km', current_s)) - current_s)` の結果を代入する。
23. `time_decay` に `math.exp(-dt_h / self.correlation_time_h)` の結果を代入する。
24. `dist_decay` に `math.exp(-ds / self.correlation_distance_km) if self.use_exp_distance_decay else 1.0 / (1.0 + ds / self.correlation_distance_km)` の結果を代入する。
25. `decay` に `time_decay * dist_decay` の結果を代入する。
26. `base` に `float(getattr(row, 'headwind_ms', 0.0))` の結果を代入する。
27. `mean` に `base + self.latest_bias * decay` の結果を代入する。
28. `sigma` に `max(self.min_sigma, math.sqrt(self.forecast_sigma0 ** 2 + self.variance_growth * dt_h) + self.measurement_sigma * (1.0 - decay))` の結果を代入する。
29. `corrected` に `mean + self.quantile_z * sigma` の結果を代入する。
30. `corrected_values.append(...)` を実行する。
31. `sigma_values.append(...)` を実行する。
32. `df['headwind_base_ms']` に `pd.to_numeric(df.get('headwind_ms', 0.0), errors='coerce').fillna(0.0)` の結果を代入する。
33. `df['headwind_mean_ms']` に `corrected_values` の結果を代入する。
34. `df['headwind_sigma_ms']` に `sigma_values` の結果を代入する。
35. `df['headwind_lower_ms']` に `df['headwind_mean_ms'] - self.confidence_z * df['headwind_sigma_ms']` の結果を代入する。
36. `df['headwind_upper_ms']` に `df['headwind_mean_ms'] + self.confidence_z * df['headwind_sigma_ms']` の結果を代入する。
37. `df['headwind_ms']` に `corrected_values` の結果を代入する。
38. `out` に `df.copy()` の結果を代入する。
39. 条件 `pd.api.types.is_datetime64_any_dtype(out['time'])` を判定し、真なら内部処理を行う。
40. `out['time']` に `out['time'].dt.tz_convert(self.forecast_time_tz).dt.strftime('%Y-%m-%d %H:%M:%S')` の結果を代入する。
41. `self.corrected_csv.parent.mkdir(...)` を実行する。
42. `out.to_csv(...)` を実行する。
43. `self.pub_status.publish(...)` を実行する。

代表コード断片

```
def _step(self) -> None:
    self._load_base()
    if self.base_df is None or self.base_df.empty:
        return
    now_utc = datetime.now(timezone.utc)
    base_now = self._interp_headwind_now(now_utc)
    if math.isfinite(self.latest_headwind):
        innovation = float(self.latest_headwind - base_now)
        self.latest_bias = 0.7 * float(self.latest_bias) + 0.3 * innovation
    corrected_now = base_now + self.latest_bias
    self.pub_headwind.publish(Float32(data=float(corrected_now)))

    df = self.base_df.copy()
    if "time" not in df.columns:
        df["time"] = [now_utc + pd.Timedelta(minutes=10 * i) for i in range(len(df))]
    if "s_km" not in df.columns:
        df["s_km"] = float(self.latest_s_km) if math.isfinite(self.latest_s_km) else 0.0
    current_s = float(self.latest_s_km) if math.isfinite(self.latest_s_km) else float(df["s_km"].iloc[0])

    corrected_values = []
    sigma_values = []
    for row in df.itertuples(index=False):
        t_utc = getattr(row, "time")
        dt_h = max(0.0, (pd.Timestamp(t_utc).to_pydatetime().astimezone(timezone.utc) -
now_utc).total_seconds() / 3600.0)
        ds = abs(float(getattr(row, "s_km", current_s)) - current_s)
        time_decay = math.exp(-dt_h / self.correlation_time_h)
        dist_decay = math.exp(-ds / self.correlation_distance_km) if self.
use_exp_distance_decay else 1.0 / (1.0 + ds / self.correlation_distance_km)
        decay = time_decay * dist_decay
        base = float(getattr(row, "headwind_ms", 0.0))
        mean = base + self.latest_bias * decay
        sigma = max(self.min_sigma, math.sqrt(self.forecast_sigma0 ** 2 + self.
variance_growth * dt_h) + self.measurement_sigma * (1.0 - decay))
        corrected = mean + self.quantile_z * sigma
        corrected_values.append(corrected)
        sigma_values.append(sigma)

    ...
```

7.9 L154 関数 main

- 行範囲: L154–L159
- 所属: module 直下
- 定義: main()->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: WindCorrectionNode, destroy_node, init, shutdown, spin
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: node
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. rclpy.init(...) を実行する。
2. node に WindCorrectionNode() の結果を代入する。
3. rclpy.spin(...) を実行する。
4. node.destroy_node(...) を実行する。
5. rclpy.shutdown(...) を実行する。

代表コード断片

```
def main() -> None:
    rclpy.init()
    node = WindCorrectionNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

7.10 設定値と入力パラメータ

行	名前	既定値・補足
23	forecast_csv	
24	corrected_forecast_csv	outputs/runtime/live_forecast_corrected.csv
25	forecast_time_tz	Australia/Darwin
26	publish_period_sec	30.0
27	measurement_sigma_ms	1.0
28	correlation_distance_km	300.0
29	fallback_correlation_time_h	3.0
30	forecast_sigma0_ms	1.5
31	forecast_variance_growth_per_hour	0.05
32	planning_quantile	0.5
33	confidence_z	1.96
34	min_sigma_ms	0.2
35	preferred_source	auto
36	use_exp_distance_decay	True

7.11 ROS topic I/O

行	種別	topic	callback / 補足
---	----	-------	---------------

57	publisher	/weather/headwind_corrected_ ms	publisher
58	publisher	/weather/wind_correction_ status	publisher
59	subscription	/weather/headwind_meas_ms	self._on_headwind
60	subscription	/vehicle/s_km	self._on_s

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/wind_correction_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。